

A* ALGORITHM

Aim

To write a Python program to implement the A* search algorithm to find the shortest path from a start node to a goal node.

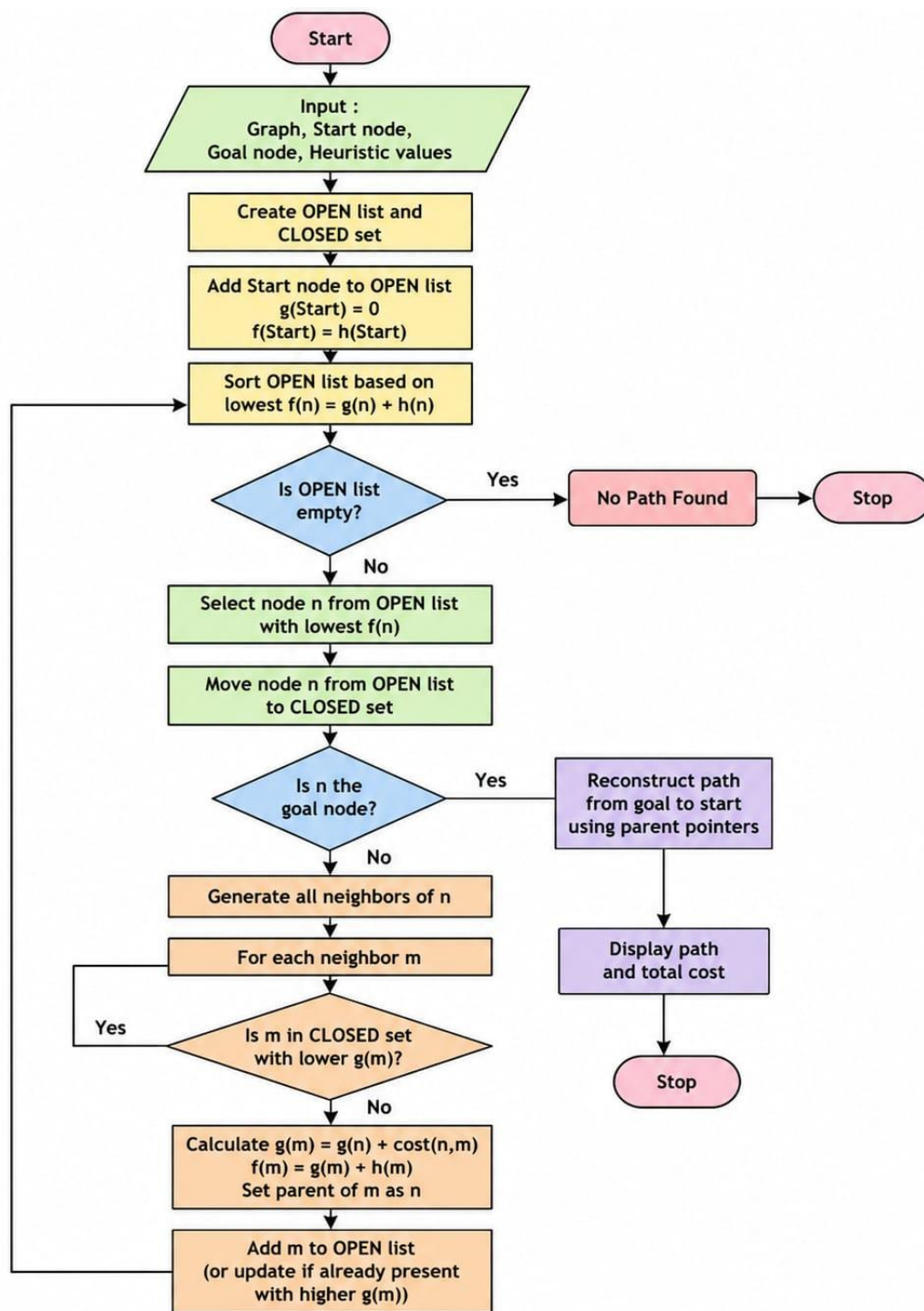
Objective

- To understand the A* search algorithm.
- To find the optimal path between the start node and the goal node.
- To use the cost function $f(n) = g(n) + h(n)$ to select the best node.
- To display the shortest path and its total cost.

Algorithm

- Start.
- Read the graph, start node, and goal node from the user.
- Initialize the open list with the start node.
- Set the cost of the start node as $g(n) = 0$.
- Calculate the evaluation function using $f(n) = g(n) + h(n)$.
- Select the node with the lowest $f(n)$ value from the open list.
- If the selected node is the goal node, display the path and total cost.
- Otherwise, expand the selected node and find its neighboring nodes.
- Calculate the $g(n)$, $h(n)$, and $f(n)$ values for each neighbor.
- Add the unvisited neighboring nodes to the open list.
- Repeat the process until the goal node is reached.
- If the goal node cannot be reached, display "No Path Found".
- Stop.

Flowchart



Python Code

```
import heapq
```

```
def astar(graph, heuristic, start, goal):
```

```

pq = [(heuristic[start], start)]
g_cost = {start: 0}
parent = {start: None}

while pq:
    f, current = heapq.heappop(pq)

    if current == goal:
        path = []
        while current is not None:
            path.append(current)
            current = parent[current]
        path.reverse()
        return path, g_cost[goal]

    for neighbor, cost in graph[current]:
        new_cost = g_cost[current] + cost

        if neighbor not in g_cost or new_cost < g_cost[neighbor]:
            g_cost[neighbor] = new_cost
            parent[neighbor] = current
            priority = new_cost + heuristic[neighbor]
            heapq.heappush(pq, (priority, neighbor))

return None, None

```

```

graph = {}

n = int(input("Enter number of nodes: "))

```

```

for _ in range(n):
    node = input("Node name: ")
    graph[node] = []

e = int(input("Enter number of edges: "))

for _ in range(e):
    u = input("Source: ")
    v = input("Destination: ")
    w = int(input("Cost: "))
    graph[u].append((v, w))
    graph[v].append((u, w))

heuristic = {}
print("\nEnter heuristic values:")
for node in graph:
    heuristic[node] = int(input(f"{node}: "))

start = input("\nEnter start node: ")
goal = input("Enter goal node: ")

path, cost = astar(graph, heuristic, start, goal)

if path:
    print("\nOptimal Path:", " -> ".join(path))
    print("Total Cost:", cost)
else:
    print("No Path Found")

```

Output

```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
===== RESTART: C:/Users/tharu/A STARTHS.py =====
Enter number of nodes: 5
Node name: A
Node name: B
Node name: C
Node name: D
Node name: E
Enter number of edges: 6
Source: A
Destination: B
Cost: 1
Source: A
Destination: C
Cost: 4
Source: B
Destination: C
Cost: 2
Source: B
Destination: D
Cost: 5
Source: C
Destination: E
Cost: 3
Source: D
Destination: E
Cost: 1

Enter heuristic values:
A: 7
B: 6
C: 2
D: 1
E: 0

Enter start node: A
Enter goal node: E

Optimal Path: A -> B -> C -> E
Total Cost: 6
```

Result

Thus, the Python program to implement the *A search algorithm** was executed successfully.

Conclusion

The A* algorithm successfully found the shortest path from the start node to the goal node by using the evaluation function $f(n) = g(n) + h(n)$. The algorithm efficiently combines the actual path cost and heuristic cost to find an optimal solution.